2026 네트워크 게임 프로그래밍

추진계획서

3D 바운스볼 – 멀티 플레이

[2022180016] [백성윤] (팀장)

[2022184008] [김성주]

[2022180026] [이도현]

목차

1. 애플리케이션 기획

1-1. 게임 정보

1-2. 게임 규칙 (멀티플레이)

1-3. 조작 방법

1-4. 개발 환경

2. High-level 디자인

2-1. 서버 high-level

2-2. 서버 - 클라이언트

1. 게임 흐름도
2. 서버 - 클라이언트 접속 단계
3. 스레드 생성

3. Low-level 디자인

3-1. 공용 헤더

3-2. 자료 구조 (Struct / Class)

3-3. 스레드 동작 흐름

3-4. 필요 함수, 기존 함수 수정

3-5. 동기화

4. 팀원별 역할 분담

5. 개발 일정

1. 애플리케이션 기획



1-1. 게임 정보

항목	내용
원본	백성윤 / 컴퓨터그래픽스 텀프로젝트
게임 제목	3D 바운스볼 – 경쟁 멀티 플레이
장르	3인 실시간 경쟁 3D 게임
목표	맵에 배치된 모든 별을 먼저 모으는 플레이어 승리
블록	바운스 / 부서지는 블록 / 가시(즉사) / 화살표(발사)
원본 기능	별 획득 시 세이브 포인트 갱신, 낙사·가시 충돌 시 리스폰, 착지 궤적 예측 표시, 1/3인칭 시점 전환, 효과음

1-2. 게임 규칙 (멀티플레이)

- 타이틀에서 서버 IP와 닉네임을 입력해 접속하고, 로비에서 3명 모두 준비하면 카운트다운 후 시작한다.
- 각 플레이어는 같은 형태의 자기 자신의 맵에서 플레이한다. → 플레이어끼리 충돌 없음, 별·부서지는 블록 상태는 플레이어마다 따로 존재한다.
- 다른 플레이어는 색이 다른 반투명 공(고스트)으로 보인다.
- 화면 좌측 상단 HUD에 3인의 닉네임과 별 개수가 표시된다.
- 게임 내 아이템 획득 시 이로운 아이템은 나에게, 해로운 아이템은 다른 모든 플레이어에게 적용한다.
- 모든 별을 제일 먼저 모은 플레이어가 승리하며, 결과 화면은 점수판을 표시하여 마무리한다.
- 게임 중 누군가 나가면 남은 인원으로 계속 진행하고, 1명만 남으면 그 플레이어가 승리한다.

1-3. 조작 방법

키	기능	비고
W A S D	공 이동	원본 유지
V	1인칭 / 3인칭 시점 전환	원본 유지
Enter	로비에서 준비 혹은 취소 (CS_READY)	추가
X	이전 세이브 포인트에서 리스폰	추가

1-4. 개발 환경

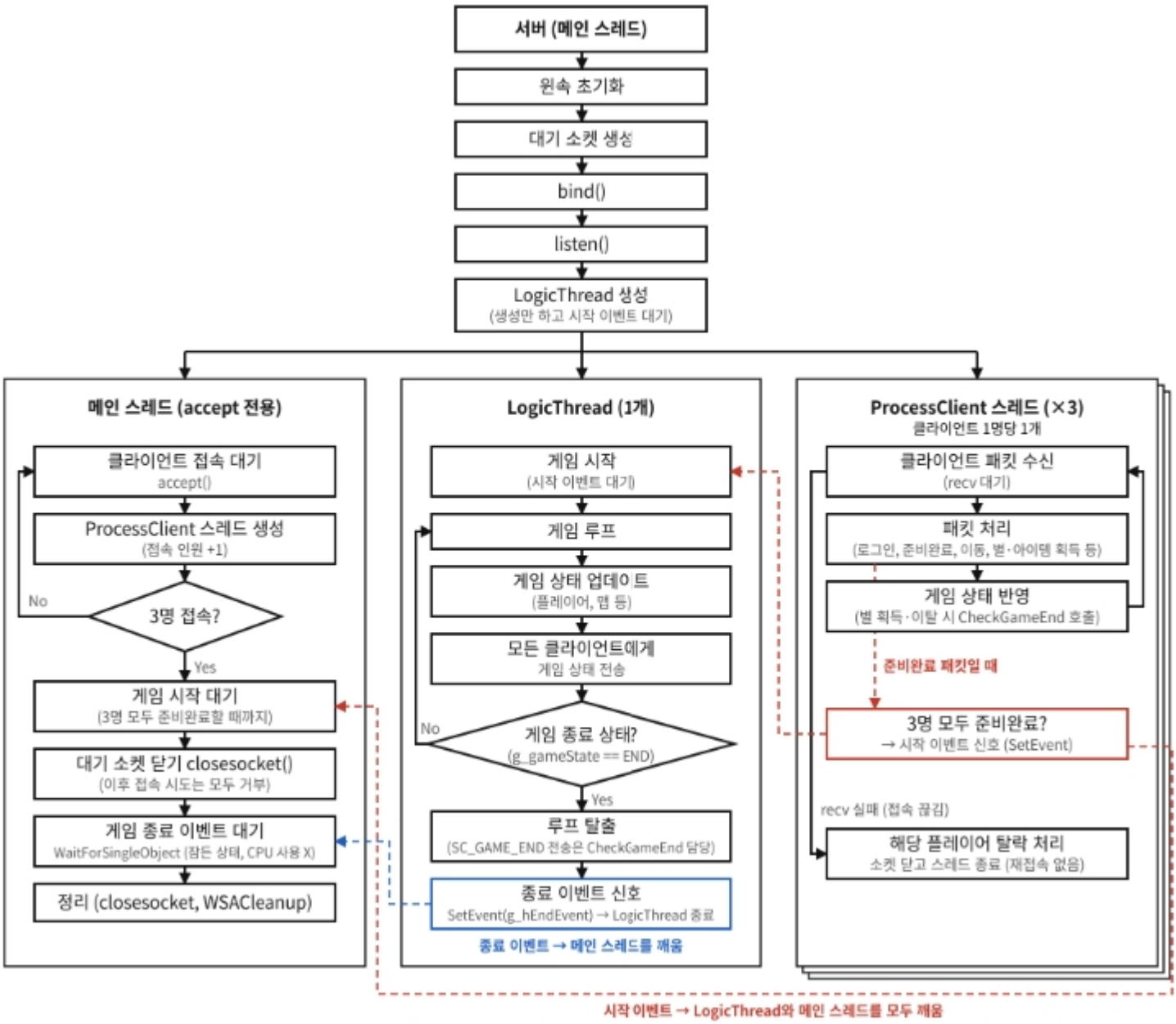
항목	내용
OS / IDE / 언어	Windows 10/11 / Visual Studio 2026 / C, C++
그래픽스	OpenGL (GLUT, GLEW, GLM, GLSL), stb_image
사운드	winmm (PlaySound)
네트워크	Winsock2 (TCP), CreateThread, CRITICAL_SECTION, Event
버전 관리	GitHub

Git 규칙

- Main 브랜치 pull / fetch → 개인 브랜치에서 작업 → 빌드 + 테스트 통과 후에만 커밋/푸시
- 커밋 형식: [날짜] 구현 내용 및 추가 설명
- 테스트: 한 PC에서 서버 1 + 클라 3 (127.0.0.1) 실행, 구현 직후 바로 테스트

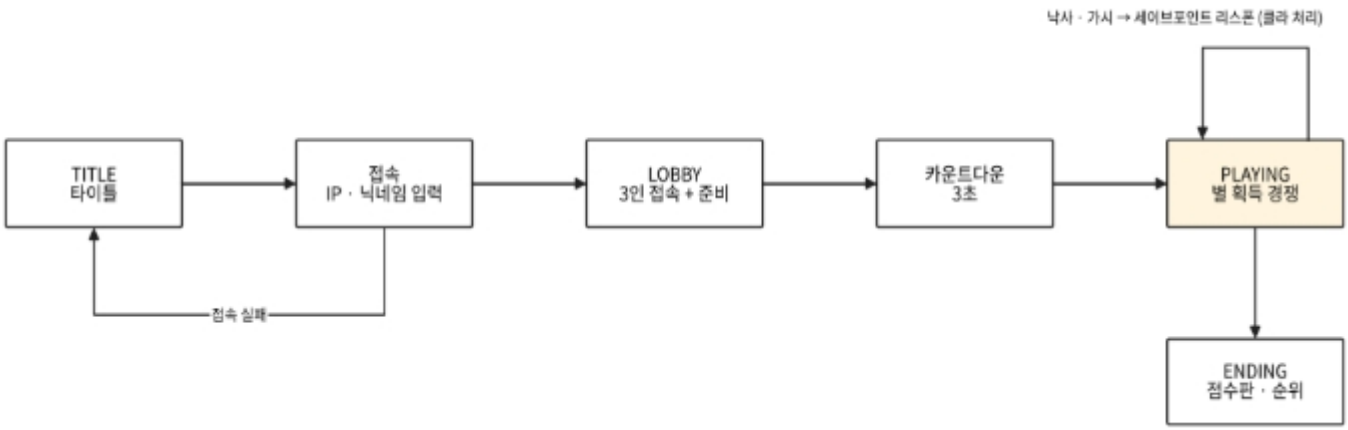
2. High-level 디자인

2-1. 서버 high-level

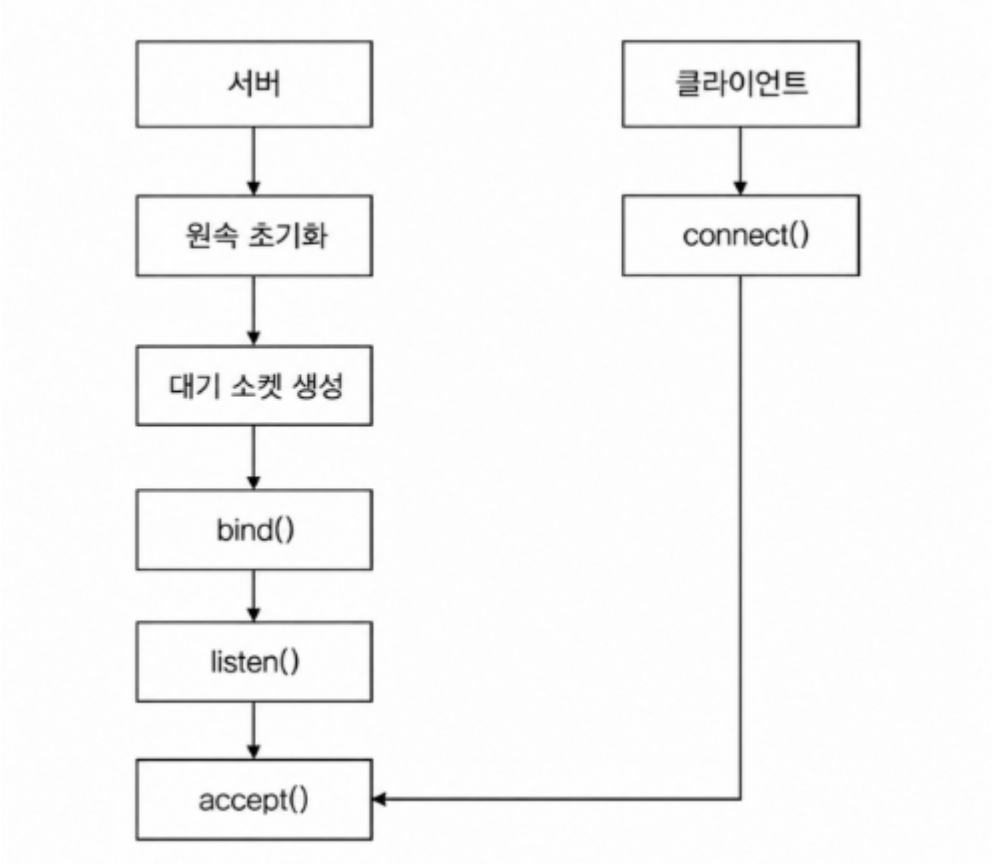


2-2. 서버 - 클라이언트

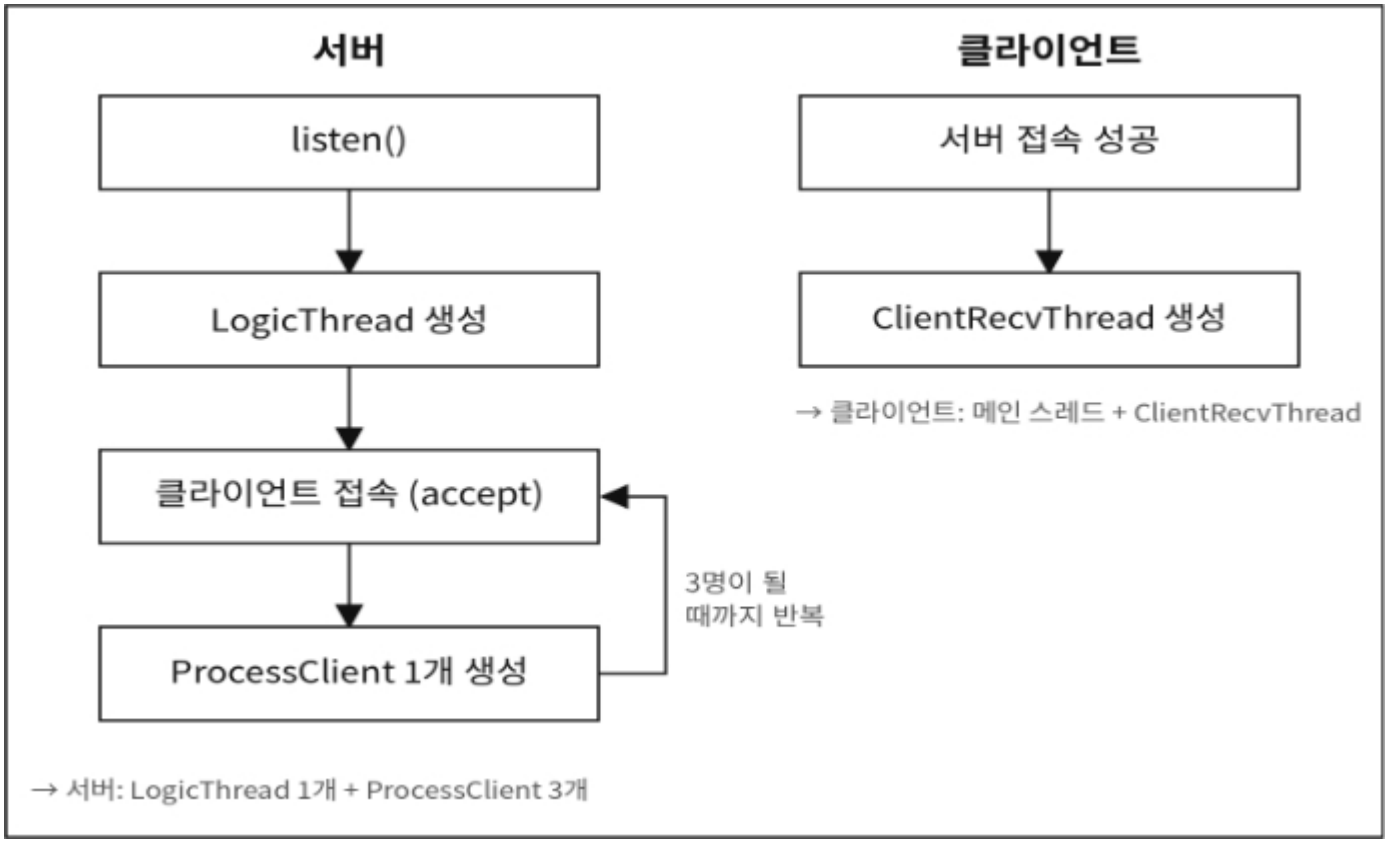
1) 게임 흐름도



2) 서버 - 클라이언트 접속 단계



3) 스레드 생성


서버
listen()
LogicThread 생성
클라이언트 접속 (accept)
ProcessClient 1개 생성
3명이 될
때까지 반복
→ 서버: LogicThread 1개 + ProcessClient 3개
클라이언트
서버 접속 성공
ClientRecvThread 생성
→ 클라이언트: 메인 스레드 + ClientRecvThread

3. Low-level 디자인

3-1. 공용 헤더

[MapData.h] 별 및 아이템 좌표

```
const unsigned int    STAR_COUNT = 7;
const float           STAR_POS[STAR_COUNT][3] = {
               { 0.0f,  0.5f,  -10.0f}, { 0.0f,  8.0f,  -45.0f}, { 0.0f,  2.0f,  -60.0f},
               {-3.0f,  5.0f,  -80.0f}, { 0.0f, 10.0f, -107.0f}, { 0.0f, 12.0f, -130.0f},
               { 0.0f, 17.0f, -143.0f} };
enum class ITEM_TYPE : unsigned char { FLYING, ENEMY_BLIND, ENEMY_SLOW };
struct ItemData { ITEM_TYPE type; float x, y, z; };
const unsigned int    ITEM_COUNT = 6;   // 임시값, 배치 확정 시 수정
const ItemData        ITEM_LIST[ITEM_COUNT] = { /* 종류·좌표 확정 시 작성 */ };
```

[Protocol.h] 패킷

패킷 = 헤더(3byte) + 본문

```
#pragma pack(push, 1)
struct PacketHeader { unsigned short size; unsigned char type; };   // 3 byte

// ... 각 패킷 struct 정의 ...
#pragma pack(pop)

enum PACKET_TYPE : unsigned char {
    CS_LOGIN = 1, CS_READY, CS_MOVE, CS_STAR_GET, CS_ITEM_GET,
    SC_LOGIN_OK, SC_PLAYER_INFO, SC_READY, SC_GAME_START, SC_GHOST,
    SC_STAR_RESULT, SC_ITEM_RESULT, SC_GAME_END, SC_LEAVE
};
```

패킷	방향	본문 (헤더 제외)	전송 시점
CS_LOGIN	C→S	char name[16]	접속 직후
CS_READY	C→S	bool isReady	로비에서 Enter
CS_MOVE	C→S	float x, y, z	초당 30번
CS_STAR_GET	C→S	char starId (0 ~ STAR_COUNT-1)	별 충돌 시
CS_ITEM_GET	C→S	char itemId	아이템 충돌 시
SC_LOGIN_OK	S→C	char id	로그인 처리 후 (요청자)
SC_PLAYER_INFO	S→C	char id; char name[16]	새 플레이어 입장 (전원)
SC_READY	S→C	char id; bool isReady	CS_READY 처리 후 (전원)
SC_GAME_START	S→C	char countdown	3인 준비 완료 (전원)
SC_GHOST	S→C	char count {char id; float x,y,z;}[3]	초당 30번 (전원)
SC_STAR_RESULT	S→C	char id, starId, starCount	GET 패킷 처리 후 (전원)
SC_ITEM_RESULT	S→C	char id, char itemID, char item_type	GET 패킷 처리 후 (전원)

패킷	방향	본문 (헤더 제외)	전송 시점
SC_GAME_END	S→C	char count {char id; char starCount;}[3]	모든 별을 모았을 시 / 1명 남았을 시 (전원)
SC_LEAVE	S→C	char id	접속 끊김 (남은 인원)

3-2. 자료 구조 (Struct / Class)

[Server]

```
class Player {
    SOCKET m_sock; int m_id; char m_name[16];
    bool m_connect, m_ready;
    float m_x, m_y, m_z;
    unsigned int n_starMask;     // 별 획득 비트마스크 (같은 별 중복 획득 방지용)
    unsigned int n_itemMask;     // 아이템 획득 비트마스크 (4바이트 -> 32개까지 가능)
    unsigned char m_starCount;
    DWORD lastStarT me;          // 동점 시 순위 판정용
};
```

[Client]

```
struct GhostInfo {
    bool  active;  char name[16];
    float prevPos[3], targePos[3];   // 보간 시작/목표
    float t;  int starCount;
};
```

```
struct NetEvent {                    // RecvThread → Main 전달용 (신규)
    char type;                       // SC_STAR_RESULT, SC_ITEM_RESULT, SC_GAME_END ...
    char data[64];                   // 패킷 본문 복사
};
```

```
struct ItemEffect {                  // 현재 적용 중인 효과 (신규)
    bool  active;
    float remainTime;                // 남은 시간, update()에서 감소
};
```

```
struct RankInfo {
    char id;
    char starCount;
};   // 엔딩 점수판
```

3-3. 스레드 동작 흐름

[Server] main() — 메인 스레드 · 1개

```
WSAStartup → socket → bind → listen
g_hStartEvent, g_hEndEvent = CreateEvent(수동 리셋)
CreateThread(LogicThread)                  // 만들자마자 시작 이벤트 대기
while (접속 인원 < 3) {
    client = accept()
    id = AssignPlayerId()                  // g_csPlayers 안에서 빈 슬롯 할당
    CreateThread(ProcessClient, id)
}
WaitForSingleObject(g_hStartEvent)         // 3명 모두 준비할 때까지 잠듦
closesocket(listen_sock)                   // 이후 접속 거부
WaitForSingleObject(g_hEndEvent)           // 게임 끝날 때까지 잠듦
남은 소켓 closesocket → 스레드 종료 대기 → WSACleanup
```

[Server] DWORD WINAPI ProcessClient(LPVOID arg) — 클라이언트당 1개 → 총 3개

```
id = (int)arg
while (true) {
    if (!RecvPacket(sock, buf)) break;     // recv 0 / SOCKET_ERROR → 끊김
    ProcessPacket(id, buf);                // type별 OnLogin / OnReady / OnMove / On*Get
}
Disconnect(id);                            // SC_LEAVE 전원 → CheckGameEnd()
return 0;
```

[Server] DWORD WINAPI LogicThread(LPVOID) — 1개

```
WaitForSingleObject(g_hStartEvent)
SC_GAME_START(countdown) 전원 → 카운트다운 → g_gameState = PLAYING
while (true) {
    g_csPlayers 안에서 3명 위치를 SC_GHOST 패킷에 복사
    SendPacketToAll(SC_GHOST)              // 임계영역 밖에서 전송
    g_csGame 안에서 end = (g_gameState == END)
    if (end) break;
    Sleep(33)                               // 초당 약 30번
}
SetEvent(g_hEndEvent)                      // 메인 스레드를 깨움
return 0;
```

[Client] main() — 메인 스레드 (GLUT) · 1개

```
IP · 닉네임 입력 → ConnectServer(ip)      // connect 후 ClientRecvThread 생성
SendLogin(name)
if (WaitForSingleObject(g_hLoginEvent, 3000) != WAIT_OBJECT_0) → 3초간 서버 답변이 없으면 접속 실패
glutTimerFunc(33, SendMove, 0);  glutMainLoop();
    update(): processNetEvents → 물리 · 충돌 → updateGhosts → 그리기
종료 시 DisconnectServer()                 // closesocket → 수신 스레드 종료 대기
```

[Client] DWORD WINAPI ClientRecvThread(LPVOID) — 1개

```
while (true) {
    if (!RecvPacket(buf)) break;           // 서버 끊김
    ProcessServerPacket(buf);
        // SC_LOGIN_OK → 내 id 저장, SetEvent(g_hLoginEvent)
        // SC_GHOST    → g_csNet 안에서 g_ghosts[] 갱신
        // 그 외       → g_csNet 안에서 NetEvent 큐에 추가
}
return 0;
```

3-4. 필요 함수, 기존 함수 수정

[Server]

함수	기능
`int AssignPlayerId()`	빈 슬롯(0~2)을 찾아 id 부여 (g_csPlayers)
`DWORD WINAPI ProcessClient(LPVOID)`	클라별 수신 루프 (스레드 함수)
`bool RecvPacket(SOCKET, char*)`	MSG_WAITALL로 헤더 → 본문 2단계 수신
`void ProcessPacket(int id, char* buf)`	패킷 type별 분기
`void OnLogin(int id, CS_LOGIN_PACKET*)`	SC_LOGIN_OK / SC_PLAYER_INFO 전송
`void OnReady(int id)`	준비 처리, SC_READY 전원 전송, 3명이면 g_hStartEvent Set
`void Disconnect(int id)`	소켓 정리, SC_LEAVE 전송, CheckGameEnd 호출
`DWORD WINAPI LogicThread(LPVOID)`	시작 대기 → 카운트다운 → 초당 30번 위치 전송 → 종료 시 g_hEndEvent Set
`void OnMove(int id, CS_MOVE_PACKET*)`	플레이어 위치 갱신
`bool CheckStar(int id, int starId)`	범위 / 중복 확인
`bool CheckItem(int id, int itemId)`	범위 / 중복 확인
`void OnStarGet(int id, CS_STAR_GET_PACKET*)`	별 개수 기록, 결과 전송, 종료 검사 호출
`void OnItemGet(int id, CS_ITEM_GET_PACKET*)`	아이템 기록, SC_ITEM_RESULT 전원 전송
`void CheckGameEnd()`	모든 별 획득 또는 1명 남음 → 종료 확정, SendGameEnd 호출
`void SendGameEnd()`	starCount, lastStarTime으로 순위를 정렬해 SC_GAME_END 전송
`void SendPacket(int id, void*, int)`	한 명에게 전송
`void SendPacketToAll(void*, int)`	접속자 전원에게 전송

[Client – Network.h / .cpp]

함수	기능
`bool ConnectServer(const char* ip)`	소켓 생성 / connect, 수신 스레드 생성
`DWORD WINAPI ClientRecvThread(LPVOID)`	서버 패킷 수신 루프 (스레드 함수)
`void ProcessServerPacket(char* buf)`	type별 분기 → SC_GHOST는 g_ghosts에 바로 쓰고, 나머지는 NetEvent 큐에 넣음
`void SendLogin(const char* name)`	CS_LOGIN 전송
`void SendReady()`	CS_READY 전송
`void SendMove(int value)`	glutTimerFunc(33) 콜백, PLAYING일 때만 내 위치 전송
`void SendStarGet(int starId)`	CS_STAR_GET 전송
`bool RecvPacket(char* buf)`	MSG_WAITALL로 헤더 → 본문 2단계 수신 (서버와 동일한 방식)
`void SendItemGet(int itemId)`	CS_ITEM_GET 전송
`void DisconnectServer()`	종료 시 closesocket, 스레드 정리, WSACleanup

[Client – GameWorld]

대상	변경 내용
`createStars()`	STAR_POS 배열로 별 생성, id 부여
`checkCollisions()`	별, 아이템 충돌 시 바로 삭제, 카운트 증가 + SendStarGet / SendItemGet 전송, 아이템은 바로 효과 적용
`onStarResult()` 신규	다른 플레이어의 별 개수 갱신
`GameState / update()`	LOBBY · COUNTDOWN 추가, ENDING은 SC_GAME_END 수신 시에만 전환
`initGhosts()` 신규	uv_sphere.obj로 고스트 모델 생성 (색 구분)
`updateGhosts(dt)` 신규	고스트 위치 선형보간
`drawGhosts()` 신규	GL_BLEND 반투명 렌더링
`drawLobby()` 신규	접속자 목록, 준비 상태, 카운트다운
`drawUI()`	3인 닉네임 + 별 개수, 적용 중인 아이템 효과 표시
`drawEndingScreen()`	승리 / 패배 문구 → 순위표
`processNetEvents()` 신규	큐를 통째로 꺼내서 type별로 onStarResult, onItemResult 등을 호출
`createItems()` 신규	ITEM_LIST로 아이템 생성, id와 type 부여
`onItemResult()` 신규	다른 플레이어가 해로운 아이템을 먹었으면 나에게 효과 적용
`startEffect(type)` / `updateEffects(dt)` 신규	효과 시작, 남은 시간 감소, 만료 시 해제 (속도·점프 값 복구)

3-5. 동기화

위치	스레드	개수
서버	Main (accept) / ProcessClient / LogicThread	1 / 3 / 1
클라	Main (glutMainLoop) / ClientRecvThread	1 / 1

동기화 객체	종류	보호 대상 / 용도
`g_csPlayers (서버)`	임계영역	g_players[] — ProcessClient ×3 ↔ LogicThread
`g_csGame (서버)`	임계영역	starMask, itemMask, starCount, g_gameState — 종료 판정 1회 보장
`g_csSend[3] (서버)`	임계영역	플레이어별 소켓 send — 여러 스레드가 같은 소켓에 동시에 보내 패킷이 섞이는 것 방지
`g_hStartEvent (서버)`	이벤트 (수동)	LogicThread가 3인 준비 완료까지 대기 (순서 제어)
`g_hEndEvent (서버)`	이벤트 (수동)	메인 스레드가 게임 종료까지 대기 — LogicThread가 END 확인 후 Set (순서 제어)
`g_csNet (클라)`	임계영역	g_ghosts[], 별 결과 큐, 게임 상태 — ClientRecvThread ↔ GLUT Main
`g_hLoginEvent (클라)`	이벤트 (자동)	glutMainLoop 진입 전 SC_LOGIN_OK 대기 (타임아웃 3초)

동기화 원칙

- 임계영역 안에서 send 호출 금지 → 데이터를 복사한 뒤 영역 밖에서 전송
- ClientRecvThread는 GL 함수 / GameWorld 객체 생성 금지 → 값만 저장하고 Main의 update()에서 반영
- GLUT 콜백 안에서 WaitForSingleObject 금지 (창이 멈춤) → 로그인 대기는 glutMainLoop 진입 전에만
- 데드락 방지: 두 락을 함께 잡아야 하면 항상 g_csPlayers → g_csGame 순서로 잡는다. g_csSend는 다른 락을 잡은 상태에서 잡지 않는다.
- 아이템 효과는 클라가 적용한다. 서버는 획득 기록과 SC_ITEM_RESULT 전원 전송만 하고, 해로운 아이템이면 받은 상대 클라가 자기에게 적용한다.

종료 판정 순서

```
bool ended = false;
EnterCriticalSection(&g_csGame);
if (g_gameState == PLAYING &&
        (starCount == STAR_COUNT || 접속 인원 <= 1)) {    // 별 모두 획득 or 1명 남음
        g_gameState = END;                                // 먼저 들어온 1명만 통과
        ended = true;
        순위 정렬(starCount 내림차순 → lastStarTime 오름차순) → 패킷에 복사;
}
LeaveCriticalSection(&g_csGame);
if (ended) SendPacketToAll(SC_GAME_END);
→ 영역 밖에서 SC_GAME_END 전송 (SendPacketToAll)
```

4. 팀원별 역할 분담

구분	구현 내용	담당자
서버	프레임워크	백성윤
	패킷 송신	김성주, 이도현
	패킷 수신	김성주, 이도현
	충돌처리	김성주
	동기화	백성윤
	추가 구현 사항 (점수 관리, 서버 연산 등)	백성윤, 이도현
클라이언트	프레임워크	이도현
	패킷 송신	김성주, 이도현
	패킷 수신	김성주, 이도현
	타이틀	백성윤
	엔딩	김성주
	추가 구현 사항 (플레이어 구분, 아이템 제작 등)	백성윤

5. 개발 일정

[백성윤] 화 수 목 금 토 일

일	월	화	수	목	금	토
10/25	**10/26**	**10/27**	**10/28**	**10/29**	**10/30** 추진계획서 제출, GitHub 저장소 생성	**10/31** [공통] Protocol.h·MapData.h 확정
11/1 서버 main: 윈속 초기화, listen/accept	**11/2**	**11/3** Player, g_players, g_csPlayers	**11/4** AssignPlayerId, ProcessClient 스레드 생성	**11/5** 3인 동시 접속 테스트	**11/6** 타이틀 화면 (IP·닉네임 입력)	**11/7** [통합] 3인 로그인
11/8 OnReady, SC_READY, g_hStartEvent	**11/9**	**11/10** LogicThread 골격 (시작 대기, 카운트다운), g_csSend	**11/11** Disconnect, SC_LEAVE	**11/12** LogicThread: SC_GHOST 30Hz 전송	**11/13** initGhosts (고스트 모델)	**11/14** [통합] 이동 동기화
11/15 updateGhosts 선형보간, drawGhosts 반투명	**11/16**	**11/17** g_csGame 적용	**11/18** CheckGameEnd, g_hEndEvent, 서버 종료 정리	**11/19** SendGameEnd 순위 정렬 (이도현과 공동 작업)	**11/20** createItems (ITEM_LIST)	**11/21** [통합] 1판 끝까지 진행
11/22 onItemResult, startEffect	**11/23**	**11/24** updateEffects, 해로운 아이템 상대 적용	**11/25** 다른 PC 테스트	**11/26** 동시 종료·이탈 동기화 테스트	**11/27** Progress Report 갱신	**11/28** [통합] 전체 시나리오
11/29 버그 수정	**11/30**	**12/1** 전송량/주기 점검	**12/2**	**12/3**	**12/4** 검수 동영상 촬영	**12/5** Progress Report 최종
12/6 최종 점검	**12/7**	**12/8** 최종 점검	**12/9** ★ 최종 제출	**12/10**	**12/11**	**12/12**

[김성주] 월 화 수 목 토 일

일	월	화	수	목	금	토
10/25	10/26	10/27	10/28	10/29	10/30	10/31 [공통] Protocol.h·MapData.h 확정 / 원본 코드 빌드 확인
11/1 SendPacket, SendPacketToAll (서버)	11/2	11/3 RecvPacket 헤더→본문 (서버)	11/4	11/5 OnMove (위치 갱신)	11/6	11/7 [통합] 3인 로그인
11/8 SendMove (glutTimerFunc 33ms)	11/9 drawLobby (접속자, 준비 상태)	11/10	11/11 checkCollisions (즉시 획득 + 서버 알림)	11/12	11/13	11/14 [통합] 이동 동기화
11/15 CheckStar (범위/중복)	11/16	11/17 OnStarGet, SC_STAR_RESULT	11/18	11/19 SendStarGet, onStarResult	11/20	11/21 [통합] 1판 끝까지 진행
11/22 CheckItem, OnItemGet	11/23	11/24 SendItemGet, SC_ITEM_RESULT	11/25	11/26 drawEndingScreen (순위표)	11/27	11/28 [통합] 전체 시나리오
11/29 SC_GAME_END → 순위표 연동	11/30	12/1	12/2 Self-Check List 작성	12/3 Self-Check List 최종	12/4	12/5 최종 점검
12/6 최종 점검	12/7 최종 점검	12/8	12/9 ★ 최종 제출	12/10	12/11	12/12

[이도현] 화 목 금 토 일

일	월	화	수	목	금	토
10/25	10/26	10/27	10/28	10/29	**10/30** 원본 코드 빌드 확인	**10/31** [공통] Protocol.h·MapData.h 확정
11/1 STAR id 추가 createStars() ConnectServer()	11/2	**11/3** ClientRecvThread(), RecvPacket()	11/4	**11/5** SendLogin() ProcessPacket() 분기 OnLogin()	**11/6** 로그인 대기 (g_hLoginEvent) → glutMainLoop g_myId 저장	**11/7** [통합] 3인 로그인
11/8 SendReady() GameState에 LOBBY·COUNTDOWN 추가	11/9	**11/10** ProcessServerPacket() 골격 SC_PLAYER_INFO, SC_READY 저장	11/11	**11/12** SC_GAME_START 처리 카운트다운 표시	**11/13** SC_GHOST 수신 → g_ghosts 갱신	**11/14** [통합] 이동 동기화
11/15 NetEvent 큐	11/16	**11/17** processNetEvents() → onStarResult·onItemResult 호출	11/18	**11/19** SendGameEnd() 순위 정렬 SC_GAME_END 형식 확정	**11/20** SC_GAME_END 수신 ENDING 전환	**11/21** [통합] 1판 끝까지 진행
11/22 서버 미실행 / 접속 실패 처리	11/23	**11/24** 이탈 고스트 제거 (SC_LEAVE)	11/25	**11/26** 다른 PC 2대 이상 접속 테스트	**11/27** DisconnectServer()	**11/28** [통합] 전체 시나리오
11/29 통합 테스트 버그 수정	11/30	**12/1** 예외 상황 검수 (이탈 · 동시 종료)	12/2	12/3	12/4	**12/5** 최종 점검
12/6 최종 점검	12/7	**12/8** 최종 점검	**12/9** ★ 최종 제출	12/10	12/11	12/12